MONASH University

The Algorithm in the Courtroom

Week 7 — Naive Bayes

Julian Garcia

FIT1061 — Introduction to Artificial Intelligence

COMMONWEALTH OF AUSTRALIA | Copyright Regulations 1969 | WARNING: This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice.

MILESTONE

COMPAS

ProPublica · *Machine Bias* · May 2016



A risk score used in US courtrooms to help decide who walks free.

What happened?

The finding. ProPublica linked ∼7,000 Broward County scores (2013–14) to two-year outcomes.

Black defendants were **∼2× as likely to be *falsely* flagged as high-risk** as white defendants who also did not reoffend.

The rebuttal. Northpointe, the vendor, replied that the tool was

equally calibrated across races — a high score meant the same reoffending rate regardless of the defendant's race.

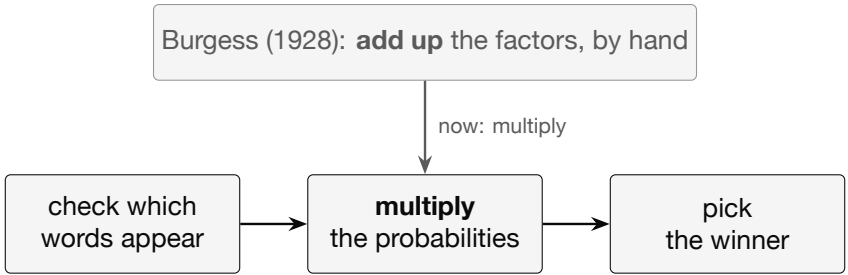
Both sides were mathematically correct.

How does a machine decide who is **“high-risk”** — and who pays when it is wrong?

This idea is nearly 100 years old



From “add up factors” to “multiply probabilities”



Same maths as a spam filter
The fix for zeros
How good is it?
Back to the courtroom

Same maths as a spam filter

the machinery, starting with a low-risk example

Last week: one word

Given the word **“free”**, flip the conditional:

$$P(\text{spam} \mid \text{“free”}) = \frac{P(\text{“free”} \mid \text{spam})\, P(\text{spam})}{P(\text{“free”})}$$

From here we write c **for the class** H (spam or ham) and w_i **for each word** in D — last week’s $P(H \mid D)$.

Why one word is not enough

A message is a **bag of words**: “free”, “prize”, “call”, “now”…

$$P(\text{free}, \text{prize}, \text{call}, \text{now} \mid \text{spam})$$

For a 1,000-word vocabulary, that is 2^{1000} combinations to count.

Why one word is not enough

A message is a **bag of words**: “free”, “prize”, “call”, “now”…

$$P(\text{free}, \text{prize}, \text{call}, \text{now} \mid \text{spam})$$

For a 1,000-word vocabulary, that is 2^{1000} combinations to count.

Impossible to count.

The naive assumption

Naive Bayes assumption

Assume the words are **independent given the class**. Then the joint becomes one probability per word, multiplied:

$$P(w_1, \ldots, w_n \mid c) \;\approx\; \prod_i P(w_i \mid c).$$

e.g. $P(\text{prize}, \text{winner} \mid \text{spam}) \approx P(\text{prize} \mid \text{spam}) \cdot P(\text{winner} \mid \text{spam})$

False in real text — but good enough to **rank** the classes.

The scoring rule

Score every class, predict the highest

$$\text{score}(c) \;=\; P(c) \prod_i P(w_i \mid c)$$

Then **predict the class with the highest score**.

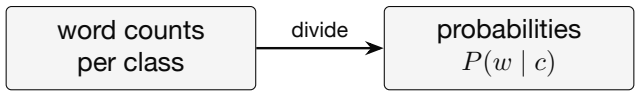
---

e.g. $\text{score}(\text{spam}) = P(\text{spam}) \cdot P(\text{free} \mid \text{spam}) \cdot P(\text{winner} \mid \text{spam})$

- $P(c)$ is the **prior** — how common the class is before reading any words.
- Denominator $P(w_1, \ldots, w_n)$: **same for every class** — cancels.
- We multiply likelihoods only for the **words that appear** in the message.
- Ties go to **ham** (the harmless class).

Training is just counting

No optimisation. No gradient descent.



Naive Bayes — the three operations

compute_priors · compute_likelihoods · predict

```
function COMPUTE_PRIORS(training):
    for each class c:
        prior[c] ← (messages in c) / (total messages)
function COMPUTE_LIKELIHOODS(training):
    for each class c, each word w in vocabulary:
        likelihood[c][w] ← (msgs in c with w) / (msgs in c)
function PREDICT(message):
    for each class c:
        score[c] ← prior[c]
        for each distinct word w present in the message and in the vocabulary:
            score[c] ← score[c] * likelihood[c][w]
    return class c with largest score[c]   # tie → ham
```

Two classes here — but the same three operations handle any number of classes: three, ten, a hundred.

Worked example — the training set

#	label	words
1	**spam**	free, prize, winner
2	**spam**	free, offer, click
3	**spam**	winner, call, now
4	ham	meeting, agenda, tuesday
5	ham	project, update, meeting
6	ham	free, lunch, friday

3 spam · 3 ham

Test message

“free meeting”

spam or ham?

Setting up the quantities

Priors: $P(\text{spam}) = P(\text{ham}) = \frac{3}{6} = 0.5$

Count the likelihoods together: how often does each word appear in each class?

Setting up the quantities

Priors: $P(\text{spam}) = P(\text{ham}) = \frac{3}{6} = 0.5$

Count the likelihoods together: how often does each word appear in each class?

	spam	ham
$P(\text{free} \mid \cdot)$	$2/3$	$1/3$
$P(\text{meeting} \mid \cdot)$	$0/3$	$2/3$

Poll

We have counted the likelihoods. One word — “meeting” — never appeared in any spam example, so its spam probability is exactly 0. We multiply all the word probabilities together to score the message.

What is the spam score?

- **A small number above zero**
- **Exactly 0**
- **About the same as the ham score**

Vote now — scan the code for your campus, *or* type the address.



Melbourne

pollev.com/juliangarcia



Malaysia

pollev.com/onghueyfang767

The verdict

$$\text{score}_{\text{spam}} = 0.5 \cdot \frac{2}{3} \cdot \mathbf{0} = \mathbf{0}$$

The verdict

$$\text{score}_{\text{spam}} = 0.5 \cdot \tfrac{2}{3} \cdot \mathbf{0} = \mathbf{0}$$

$$\text{score}_{\text{ham}} = 0.5 \cdot \tfrac{1}{3} \cdot \tfrac{2}{3} = \tfrac{1}{9}$$

Prediction: **ham**.

The zero-frequency problem

"meeting" never appeared in a spam email, so $P(\text{meeting} \mid \text{spam}) = 0$. That single zero drags the **whole product** to 0 — no matter how much other evidence points to spam.

Same maths as a spam filter

The fix for zeros

How good is it?

Back to the courtroom

The fix for zeros

pretend you saw a little of everything

Laplace smoothing

The zero fix

$$P(w \mid c) \;=\; \frac{\text{count}(w, c) + 1}{\text{count}(c) + 2}.$$

$\text{count}(w, c)$ = messages in c containing w; $\text{count}(c)$ = messages in c.

- The $+1$ gives every word one small imaginary observation — so nothing is ever exactly zero.
- The $+2$ reflects the two things that could be counted for each word: *seen* or *not seen* in the class.



Pierre-Simon Laplace

(1749–1827)

Re-run the verdict — with smoothing

Same “free meeting” test, now smoothed:

$$P(\text{meeting} \mid \text{spam}) = \frac{0+1}{3+2} = \frac{1}{5} \quad \textit{(no longer zero)} \qquad P(\text{free} \mid \text{spam}) = \frac{2+1}{3+2} = \frac{3}{5}$$

$$P(\text{free} \mid \text{ham}) = \frac{1+1}{3+2} = \frac{2}{5} \qquad P(\text{meeting} \mid \text{ham}) = \frac{2+1}{3+2} = \frac{3}{5}$$

$$\text{score}_{\text{spam}} = 0.5 \cdot \frac{3}{5} \cdot \frac{1}{5} = 0.06 \qquad \text{score}_{\text{ham}} = 0.5 \cdot \frac{2}{5} \cdot \frac{3}{5} = 0.12$$

Key Takeaway

Still **ham** — but spam is **a real possibility again**.

Feature selection

Not every word helps

Remove stop-words (“the”, “a”, “is”); keep the top-k most informative words. A smaller, well-chosen vocabulary can beat the full one.

In our trace, “free” pulls the classes apart ($2/3$ vs $1/3$) — it **helps**.
A word in *every* message, spam and ham alike, pulls nothing apart — it only adds noise.

Informative $=$ far more common in one class than the other;
rank words by $|P(w \mid \text{spam}) - P(w \mid \text{ham})|$.

From products to sums — using logs

For interest — not required in this week's lab.

A different numerical problem — not zeros, but *many tiny numbers*.

Multiplying ∼50 tiny probabilities **underflows**:

$$0.01^{50} \to \mathbf{0.0} \text{ in floating point,}$$
$$\text{though the true value is } 10^{-100}.$$

Fix: **add logs** instead of multiplying
($\log$ = natural log).

$$\log(a \cdot b) = \log a + \log b$$
$$50 \cdot \log(0.01) = -230.3 = \log(10^{-100})$$



Each word's $\log P(w \mid c)$ is its **contribution** — see which words contributed most to the decision.

Same maths as a spam filter

The fix for zeros

How good is it?

Back to the courtroom

How good is it?

measuring the classifier

The confusion matrix

	predicted spam	predicted ham
actual spam	**TP** spam caught	**FN** spam the filter missed
actual ham	**FP** ham wrongly flagged	**TN** ham correctly kept

Confusion matrix

A 2×2 tally of predictions against the truth, split into **true** vs **false** for each class.

Precision and recall

Precision

$$\frac{\text{TP}}{\text{TP} + \text{FP}}$$

Of the messages we flagged spam, how many really were?

Recall

$$\frac{\text{TP}}{\text{TP} + \text{FN}}$$

Of the real spam, how many did we catch?

Overall **accuracy** $= \dfrac{\text{TP} + \text{TN}}{\text{total}}$ — the share of all messages classified correctly.

Here spam is the positive class — TP means a real spam correctly flagged.

Scoring it on eight unseen (held-out) messages

eight held-out messages: 3 spam, 5 ham.

	predicted spam	predicted ham
actual spam	TP = 2	FN = 1
actual ham	FP = 0	TN = 5

$$\text{precision} = \frac{2}{2+0} = 100\%$$

$$\text{recall} = \frac{2}{2+1} = 67\%$$

$$\text{accuracy} = \frac{2+5}{8} = 88\%$$

The message it missed

“free msg claim your reward call the number below”

→ classified **ham**: a real spam message the filter missed — a **false negative**.

Poll

A spam filter can make two kinds of mistake: a real email is sent to the spam folder, or a spam email reaches your inbox.

For a spam filter, which mistake is worse?

- **Sending a real email to the spam folder**
- **Letting a spam email reach the inbox**
- **They are equally bad**

Vote now — scan the code for your campus, *or* type the address.



Melbourne

pollev.com/juliangarcia



Malaysia

pollev.com/onghueyfang767

Which error is worse?

For a spam filter

A **false positive** (a real email sent to junk you may never see) hurts more than a **false negative** (spam that reaches your inbox: annoying, but visible). So we tune for **high precision**.

Which error is worse depends **entirely on what is at risk**.

Same maths as a spam filter

The fix for zeros

How good is it?

Back to the courtroom

Back to the courtroom

now replace emails with people

The cost asymmetry

	Spam classifier	COMPAS
input	words in a message	age, prior arrests, employment…
output	spam / ham	high-risk / low-risk
false positive	real email deleted	**flagged high-risk, detained — never re-arrested**
false negative	spam reaches inbox *(annoying)*	someone later re-arrested is released
stakes	convenience	**liberty**

A rule short enough to read aloud

COMPAS

- 137 questionnaire items
- closed-source
- $\sim$65% accurate
  (always “low-risk” $\approx$ 55%)

A rule built from the same data

“More than 3 prior offences, OR 18–20 and male, OR 21–23 with $\geq$2 priors $\rightarrow$ predict re-arrest; otherwise not.”

In one study, a 2–3 factor rule matched COMPAS’s $\sim$65% accuracy (Dressel & Farid, 2018).

So why is the one deciding bail the one **no one can inspect?**

Both were telling the truth

Northpointe

“equally calibrated.”

ProPublica

“unequal false-positive rates across groups.”

There is more than one definition of **“fair”** —
and they cannot all hold at once.

We make this precise in a few weeks.

Poll

Before an AI system is allowed to help set bail, which safeguard matters most to you?

- The right to inspect how it decides
- A named person accountable when it is wrong
- Proof it is not worse for some groups
- It should not be used at all

Vote now — scan the code for your campus, *or* type the address.



Melbourne

pollev.com/juliangarcia



Malaysia

pollev.com/onghueyfang767

What safeguards would you demand?

Discussion

What safeguards would you demand before an AI system helps **set bail**?

Consider: the right to inspect the reasoning; who is accountable for an error; which error the system is tuned to avoid; whether the data measures crime or measures policing.

You will write 150–250 words on this.

This week’s debate

Motion

Risk scores have no place in bail decisions.

Take a side on Kialo (a debate app) — for or against.



Scan to join — or open:

kialo-edu.com/p/d1f78d4b-0578-4114-b8e6-58f392a5b28c/709785

Recap

Idea	What	Why it matters
naive independence	multiply one probability per word	makes the count feasible
zero-frequency	one unseen word zeros the product	a product cannot recover
Laplace smoothing	add $+1$ / $+2$ imaginary counts	no catastrophic zero
log-sums (optional)	add logs instead of multiplying	avoids underflow
confusion matrix precision & recall	TP / FP / FN / TN tally false alarms vs misses	separates the error types accuracy alone hides the trade-off

In the lab this week

Pass tier. **Aiming for a Pass**

1. Build the many-word classifier: count, multiply prior × likelihoods, pick the higher score.
2. Watch a single zero collapse a score.
3. Measure it with a confusion matrix, precision, and recall.

Credit tier. **Aiming for a Credit** — all of the above, plus:

- Fix the zeros with **Laplace smoothing**.
- Choose your features instead of using every word.

Distinction tier. **No Distinction task** — but an idea to carry forward:

- When two groups get different error rates, **which definition of "fair" do you optimise — and what do you give up?**

This week, the machine **counted**.

It never adjusted a single number.

This week, the machine **counted**.

It never adjusted a single number.

Next week: a machine that **tunes its own numbers**

to fit the data — the first step toward **learning**.